13
COMPONENT COHESION

Which classes belong in which components? This is an important decision, and requires guidance from good software engineering principles. Unfortunately, over the years, this decision has been made in an ad hoc manner based almost entirely on context.

In this chapter we will discuss the three principles of component cohesion:

- **REP:** The Reuse/Release Equivalence Principle
- **CCP:** The Common Closure Principle
- **CRP:** The Common Reuse Principle

THE REUSE/RELEASE EQUIVALENCE PRINCIPLE

The granule of reuse is the granule of release.

The last decade has seen the rise of a menagerie of module management tools, such as Maven, Leiningen, and RVM. These tools have grown in importance because, during that time, a vast number of reusable components and component libraries have been created. We are now living in the age of software reuse—a fulfillment of one of the oldest promises of the object-oriented model.

The Reuse/Release Equivalence Principle (REP) is a principle that seems obvious, at least in hindsight. People who want to reuse software components cannot, and will not, do so unless those components are tracked through a release process and are given release numbers.

This is not simply because, without release numbers, there would be no way to ensure that all the reused components are compatible with each other. Rather, it also reflects the fact that software developers need to know when new releases are coming, and which changes those new releases will bring.

It is not uncommon for developers to be alerted about a new release and decide, based on the changes made in that release, to continue to use the old release instead. Therefore the release process must produce the appropriate notifications and release documentation so that users can make informed decisions about when and whether to integrate the new release.

From a software design and architecture point of view, this principle means that the classes and modules that are formed into a component must belong to a cohesive group. The component cannot simply consist of a random hodgepodge of classes and modules; instead, there must be some overarching theme or purpose that those modules all share.

Of course, this should be obvious. However, there is another way to look at this issue that is perhaps not quite so obvious. Classes and modules that are grouped together into a component should be *releasable* together. The fact that they share the same version number and the same release tracking, and are included under the same release documentation, should make sense both to the author and to the users.

This is weak advice: Saying that something should “make sense” is just a way of

waving your hands in the air and trying to sound authoritative. The advice is weak because it is hard to precisely explain the glue that holds the classes and modules together into a single component. Weak though the advice may be, the principle itself is important, because violations are easy to detect—they don't "make sense." If you violate the REP, your users will know, and they won't be impressed with your architectural skills.

The weakness of this principle is more than compensated for by the strength of the next two principles. Indeed, the CCP and the CRP strongly define the this principle, but in a negative sense.

THE COMMON CLOSURE PRINCIPLE

> *Gather into components those classes that change for the same reasons and at the same times. Separate into different components those classes that change at different times and for different reasons*.

This is the Single Responsibility Principle restated for components. Just as the SRP says that a *class* should not contain multiples reasons to change, so the Common Closure Principle (CCP) says that a *component* should not have multiple reasons to change.

For most applications, maintainability is more important than reusability. If the code in an application must change, you would rather that all of the changes occur in one component, rather than being distributed across many components.[1] If changes are confined to a single component, then we need to redeploy only the one changed component. Other components that don't depend on the changed component do not need to be revalidated or redeployed.

The CCP prompts us to gather together in one place all the classes that are likely to change for the same reasons. If two classes are so tightly bound, either physically or conceptually, that they always change together, then they belong in the same component. This minimizes the workload related to releasing, revalidating, and redeploying the software.

This principle is closely associated with the Open Closed Principle (OCP). Indeed, it is "closure" in the OCP sense of the word that the CCP addresses. The OCP states that classes should be closed for modification but open for extension. Because 100% closure is not attainable, closure must be strategic. We design our classes such that they are closed to the most common kinds of changes that we expect or have

experienced.

The CCP amplifies this lesson by gathering together into the same component those classes that are closed to the same types of changes. Thus, when a change in requirements comes along, that change has a good chance of being restricted to a minimal number of components.

SIMILARITY WITH SRP

As stated earlier, the CCP is the component form of the SRP. The SRP tells us to separate methods into different classes, if they change for different reasons. The CCP tells us to separate classes into different components, if they change for different reasons. Both principles can be summarized by the following sound bite:

> *Gather together those things that change at the same times and for the same reasons. Separate those things that change at different times or for different reasons.*

THE COMMON REUSE PRINCIPLE

> *Don't force users of a component to depend on things they don't need.*

The Common Reuse Principle (CRP) is yet another principle that helps us to decide which classes and modules should be placed into a component. It states that classes and modules that tend to be reused together belong in the same component.

Classes are seldom reused in isolation. More typically, reusable classes collaborate with other classes that are part of the reusable abstraction. The CRP states that these classes belong together in the same component. In such a component we would expect to see classes that have lots of dependencies on each other.

A simple example might be a container class and its associated iterators. These classes are reused together because they are tightly coupled to each other. Thus they ought to be in the same component.

But the CRP tells us more than just which classes to put together into a component: It also tells us which classes *not* to keep together in a component. When one component uses another, a dependency is created between the components. Perhaps the *using* component uses only one class within the *used* component—but that still doesn't weaken the dependency. The *using* component still depends on the *used* component.

Because of that dependency, every time the *used* component is changed, the *using* component will likely need corresponding changes. Even if no changes are necessary to the *using* component, it will likely still need to be recompiled, revalidated, and redeployed. This is true even if the *using* component doesn't care about the change made in the *used* component.

Thus when we depend on a component, we want to make sure we depend on every class in that component. Put another way, we want to make sure that the classes that we put into a component are inseparable—that it is impossible to depend on some and not on the others. Otherwise, we will be redeploying more components than is necessary, and wasting significant effort.

Therefore the CRP tells us more about which classes *shouldn't* be together than about which classes *should* be together. The CRP says that classes that are not tightly bound to each other should not be in the same component.

RELATION TO ISP

The CRP is the generic version of the ISP. The ISP advises us not to depend on classes that have methods we don't use. The CRP advises us not to depend on components that have classes we don't use.

All of this advice can be reduced to a single sound bite:

> *Don't depend on things you don't need.*

THE TENSION DIAGRAM FOR COMPONENT COHESION

You may have already realized that the three cohesion principles tend to fight each other. The REP and CCP are *inclusive* principles: Both tend to make components larger. The CRP is an *exclusive* principle, driving components to be smaller. It is the tension between these principles that good architects seek to resolve.

Figure 13.1 is a tension diagram[2] that shows how the three principles of cohesion interact with each other. The edges of the diagram describe the *cost* of abandoning the principle on the opposite vertex.

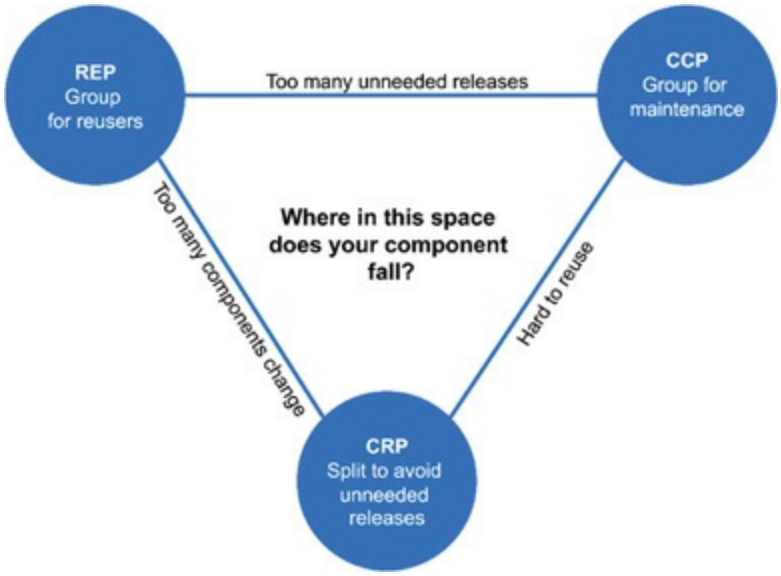

Figure 13.1 Cohesion principles tension diagram

An architect who focuses on just the REP and CRP will find that too many components are impacted when simple changes are made. In contrast, an architect who focuses too strongly on the CCP and REP will cause too many unneeded releases to be generated.

A good architect finds a position in that tension triangle that meets the *current* concerns of the development team, but is also aware that those concerns will change over time. For example, early in the development of a project, the CCP is much more important than the REP, because develop-ability is more important than reuse.

Generally, projects tend to start on the right hand side of the triangle, where the only sacrifice is reuse. As the project matures, and other projects begin to draw from it, the project will slide over to the left. This means that the component structure of a project can vary with time and maturity. It has more to do with the way that project is developed and used, than with what the project actually does.

CONCLUSION

In the past, our view of cohesion was much simpler than the REP, CCP, and CRP implied. We once thought that cohesion was simply the attribute that a module performs one, and only one, function. However, the three principles of component cohesion describe a much more complex variety of cohesion. In choosing the classes to group together into components, we must consider the opposing forces involved in reusability and develop-ability. Balancing these forces with the needs of the

application is nontrivial. Moreover, the balance is almost always dynamic. That is, the partitioning that is appropriate today might not be appropriate next year. As a consequence, the composition of the components will likely jitter and evolve with time as the focus of the project changes from develop-ability to reusability.

1. See the section on “The Kitty Problem” in Chapter 27, “Services: Great and Small.“
2. Thanks to Tim Ottinger for this idea.